

CS 224N Winter 2026 Assignment 3

Self-Attention and Transformers

Báo cáo này trình bày lời giải chi tiết cho ba phần của Assignment 3: (1) khám phá cơ chế self-attention, (2) các tính chất của position embeddings, và (3) hiện thực một Transformer kiểu GPT-2 từ đầu cùng kết quả huấn luyện.

1. Attention Exploration (14 points)

Nhắc lại định nghĩa toán học của cơ chế attention trên một vector truy vấn (query) $\mathbf{q} \in \mathbb{R}^d$, tập hợp các vector giá trị (values) $\{\mathbf{v}_1, \dots, \mathbf{v}_n\}$ và tập hợp các vector khóa (keys) $\{\mathbf{k}_1, \dots, \mathbf{k}_n\}$:

$$\mathbf{c} = \sum_{i=1}^n \alpha_i \mathbf{v}_i, \quad \alpha_i = \frac{\exp(\mathbf{k}_i^\top \mathbf{q})}{\sum_{j=1}^n \exp(\mathbf{k}_j^\top \mathbf{q})}$$

Question 1(a) — Copying in attention

- i. Phân phối xác suất α sẽ dồn gần như toàn bộ khối lượng vào một thành phần α_j duy nhất khi tích vô hướng giữa query và key tại vị trí j lớn hơn vượt trội so với tất cả các vị trí còn lại trong từ điển. Điều kiện toán học cụ thể là:

$$\mathbf{k}_j^\top \mathbf{q} \gg \mathbf{k}_i^\top \mathbf{q} \quad \forall i \neq j$$

Hiện tượng này xảy ra khi vector truy vấn $\mathbf{q}$ có hướng gần như trùng khít (cùng phương, cùng chiều) và có độ dài đủ lớn để “trở” trực tiếp vào riêng vector khóa $\mathbf{k}_j$, trong khi gần như trực giao với tất cả các khóa $\mathbf{k}_i$ khác.

- ii. Khi điều kiện trên được thỏa mãn, trọng số attention tại vị trí j tiến gần đến giá trị tuyệt đối $\alpha_j \approx 1$ và các trọng số còn lại triệt tiêu về mức $\alpha_{i \neq j} \approx 0$. Khi đó, biểu thức tính vector ngữ cảnh đầu ra được khai triển như sau:

$$\mathbf{c} = \sum_{i=1}^n \alpha_i \mathbf{v}_i \approx \mathbf{v}_j$$

Hệ quả này minh chứng cho cơ chế sao chép (“copying”): lớp attention trích xuất và tái tạo lại nguyên vẹn vector giá trị $\mathbf{v}_j$ ở đầu ra.

Question 1(b) — An average of two

Giả thiết các vector khóa trong hệ thống trực giao đôi một ($\mathbf{k}_i^\top \mathbf{k}_j = 0$ với mọi $i \neq j$) và tuân thủ chuẩn hóa đơn vị ($\|\mathbf{k}_i\|_2 = 1$). Để đầu ra thu được là trung bình công của hai vector giá trị mục tiêu, tức $\mathbf{c} \approx \frac{1}{2}(\mathbf{v}_a + \mathbf{v}_b)$, ta thiết lập vector truy vấn $\mathbf{q}$ theo công thức:

$$\mathbf{q} = M(\mathbf{k}_a + \mathbf{k}_b)$$

Trong đó, $M \gg 0$ là một hằng số dương có giá trị cực lớn. Áp dụng tính chất trực giao và chuẩn đơn vị, ta tính toán tích vô hướng cho từng trường hợp:

$$\mathbf{k}_a^\top \mathbf{q} = M, \quad \mathbf{k}_b^\top \mathbf{q} = M, \quad \mathbf{k}_i^\top \mathbf{q} = 0 \quad \forall i \neq a, b$$

Thay các giá trị tích vô hướng này vào hàm Softmax để xác định trọng số chú ý:

$$\alpha_a = \alpha_b = \frac{e^M}{2e^M + (n-2)e^0} \xrightarrow{M \rightarrow \infty} \frac{1}{2}, \quad \alpha_{i \neq a, b} \rightarrow 0$$

Do đó, vector đầu ra hội tụ về giá trị $\mathbf{c} \approx \frac{1}{2}(\mathbf{v}_a + \mathbf{v}_b)$. Về mặt trực giác, việc đặt $\mathbf{q}$ là tổng của hai vector khóa mục tiêu kết hợp hệ số khuếch đại lớn sẽ ép hàm Softmax bão hòa và phân phối đều trọng số chú ý cho riêng hai vị trí a và b .

Question 1(c) — Drawbacks of single-headed attention

Xét trường hợp các vector khóa là các biến ngẫu nhiên tuân theo phân phối chuẩn $\mathbf{k}_i \sim \mathcal{N}(\boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i)$, với các vector kỳ vọng $\boldsymbol{\mu}_i$ trực giao và có chuẩn đơn vị.

- i. Khi các ma trận hiệp phương sai có dạng $\boldsymbol{\Sigma}_i = \alpha \mathbf{I}$ với $\alpha \rightarrow 0^+$, các vector khóa biến đổi thành các đại lượng gần như tất định ($\mathbf{k}_i \approx \boldsymbol{\mu}_i$). Ta chọn vector truy vấn tương tự phần trước:

$$\mathbf{q} = M(\boldsymbol{\mu}_a + \boldsymbol{\mu}_b), \quad M \gg 0$$

Khi đó, ta có $\mathbf{k}_a^\top \mathbf{q} \approx M$, $\mathbf{k}_b^\top \mathbf{q} \approx M$, và $\mathbf{k}_i^\top \mathbf{q} \approx 0$ với mọi $i \neq a, b$. Theo lập luận toán học ở câu 1(b), ta thu được $\alpha_a \approx \alpha_b \approx \frac{1}{2}$, giúp đầu ra ổn định ở giá trị trung bình $\mathbf{c} \approx \frac{1}{2}(\mathbf{v}_a + \mathbf{v}_b)$ do phương sai vô cùng nhỏ.

- ii. Xét cấu hình nhiễu bất đối xứng: $\boldsymbol{\Sigma}_a = \alpha \mathbf{I} + \frac{1}{2} \boldsymbol{\mu}_a \boldsymbol{\mu}_a^\top$ ($\alpha \rightarrow 0^+$) và $\boldsymbol{\Sigma}_i = \alpha \mathbf{I}$ với mọi $i \neq a$. Lúc này, vector khóa $\mathbf{k}_a$ có xu hướng giữ nguyên hướng $\boldsymbol{\mu}_a$ nhưng độ lớn biên độ chịu dao động mạnh, được biểu diễn qua biến ngẫu nhiên vô hướng $\eta \sim \mathcal{N}(0, \frac{1}{2})$ như sau: $\mathbf{k}_a \approx (1 + \eta) \boldsymbol{\mu}_a$. Tính toán tích vô hướng với query $\mathbf{q}$:

$$\mathbf{k}_a^\top \mathbf{q} \approx M(1 + \eta), \quad \mathbf{k}_b^\top \mathbf{q} \approx M$$

Do hằng số M có giá trị rất lớn, chỉ cần một dao động vi mô của η cũng bị khuếch đại thành một khoảng chênh lệch khổng lồ $M\eta$ trong số mũ của hàm Softmax:

- **Khi $\eta > 0$:** Ta có $\exp(\mathbf{k}_a^\top \mathbf{q}) \gg \exp(\mathbf{k}_b^\top \mathbf{q})$, dẫn đến $\alpha_a \approx 1$ và đầu ra suy biến thành $\mathbf{c} \approx \mathbf{v}_a$.
- **Khi $\eta < 0$:** Ta có α_b áp đảo, dẫn đến $\alpha_b \approx 1$ và đầu ra suy biến thành $\mathbf{c} \approx \mathbf{v}_b$.

Hệ quả là qua các phiên lấy mẫu ngẫu nhiên khác nhau, đầu ra $\mathbf{c}$ liên tục nhảy bật qua lại giữa hai trạng thái cực đoan $\mathbf{v}_a$ và $\mathbf{v}_b$ thay vì duy trì ở giá trị trung bình mong muốn, khiến phương sai của $\mathbf{c}$ rất lớn. Đây là nhược điểm chí mạng của single-headed attention khi một đầu chú ý duy nhất buộc phải cân bằng hai khóa cạnh tranh trong cùng một không gian Softmax.

Question 1(d) — Benefits of multi-headed attention

Xét kiến trúc Multi-headed Attention gồm hai đầu độc lập sinh ra hai vector ngữ cảnh $\mathbf{c}_1, \mathbf{c}_2$, với đầu ra tổng hợp được tính theo cơ chế trung bình: $\mathbf{c} = \frac{1}{2}(\mathbf{c}_1 + \mathbf{c}_2)$.

- i. Với điều kiện không nhiễu $\boldsymbol{\Sigma}_i = \alpha \mathbf{I}$ ($\alpha \rightarrow 0^+$), ta phân tách nhiệm vụ bằng cách gán cho mỗi đầu một vector truy vấn riêng biệt, tập trung vào đúng một khóa mục tiêu:

$$\mathbf{q}_1 = M \boldsymbol{\mu}_a, \quad \mathbf{q}_2 = M \boldsymbol{\mu}_b, \quad M \gg 0$$

Lúc này, đầu thứ nhất chỉ bão hòa tại khóa a ($\mathbf{c}_1 \approx \mathbf{v}_a$) và đầu thứ hai chỉ bão hòa tại khóa b ($\mathbf{c}_2 \approx \mathbf{v}_b$), mang lại kết quả tổng hợp chính xác: $\mathbf{c} \approx \frac{1}{2}(\mathbf{v}_a + \mathbf{v}_b)$.

ii. Xét điều kiện nhiều biên độ cao tại vị trí a ($\Sigma_a = \alpha \mathbf{I} + \frac{1}{2} \mu_a \mu_a^\top$). Khi áp dụng cùng hệ thống query $\mathbf{q}_1, \mathbf{q}_2$ như trên:

- **Tại đầu chú ý 1:** Tích vô hướng đạt $\mathbf{k}_a^\top \mathbf{q}_1 \approx M(1 + \eta)$. Dù biến ngẫu nhiên η nhận giá trị âm hay dương, với điều kiện M cực lớn, giá trị $M(1 + \eta)$ vẫn luôn dương và vượt trội hoàn toàn so với mức nhiễu của các khóa khác ($\mathbf{k}_i^\top \mathbf{q}_1 \approx 0$), nên $\alpha_a \approx 1$ và đầu ra $\mathbf{c}_1 \approx \mathbf{v}_a$ luôn ổn định với phương sai cực thấp.
- **Tại đầu chú ý 2:** Query $\mathbf{q}_2$ chỉ nhắm vào mục tiêu μ_b . Do ma trận hiệp phương sai tại vị trí này không bị nhiễu biên độ ($\Sigma_b = \alpha \mathbf{I}$), đầu ra $\mathbf{c}_2 \approx \mathbf{v}_b$ đạt độ ổn định tuyệt đối.

Tổng hợp lại, vector ngữ cảnh cuối cùng $\mathbf{c} = \frac{1}{2}(\mathbf{c}_1 + \mathbf{c}_2) \approx \frac{1}{2}(\mathbf{v}_a + \mathbf{v}_b)$ đạt được độ chính xác cao và triệt tiêu được hoàn toàn biến động phương sai qua các phiên lấy mẫu.

Question 1(e) — Summary of Head Mechanisms

Cơ chế multi-headed attention khắc phục triệt để nhược điểm của single-headed bằng cách **giao cho mỗi đầu chú ý một không gian truy vấn độc lập nhắm tới đúng một khóa mục tiêu**. Thiết kế này cho phép mỗi đầu chuyên biệt hóa nhiệm vụ, chỉ tập trung bão hòa hàm Softmax để sao chép duy nhất một vector giá trị mục tiêu, giúp cô lập và miễn nhiễm hoàn toàn trước các biến động về độ lớn hay nhiễu biên độ của các vector khóa; phép toán trung bình cộng ở giai đoạn cuối cùng giúp khôi phục lại trạng thái thông tin đa ngữ cảnh với mức phương sai cực thấp. Ngược lại, kiến trúc single-headed bắt buộc một query duy nhất phải xử lý đồng thời nhiều khóa cạnh tranh trong cùng một phân phối Softmax, khiến hệ thống cực kỳ nhạy cảm trước các biến động nhiễu biên độ, gây ra hiện tượng mất ổn định đầu ra.

2. Position Embeddings Exploration (6 points)

Question 2(a) — Permuting the input

- i. Xét ma trận đầu vào bị biến đổi bởi ma trận hoán vị $\mathbf{P}$ ($\mathbf{P}^\top \mathbf{P} = \mathbf{I}, \mathbf{P}\mathbf{1} = \mathbf{1}$) dạng $\mathbf{X}_{\text{perm}} = \mathbf{P}\mathbf{X}$. Ta chứng minh ma trận đầu ra thỏa mãn $\mathbf{Z}_{\text{perm}} = \mathbf{P}\mathbf{Z}$ qua hai giai đoạn:

Giai đoạn 1: Lớp toán học Self-Attention. Các ma trận Query, Key, Value biến đổi tuyến tính:

$$\mathbf{Q}_{\text{perm}} = \mathbf{X}_{\text{perm}} \mathbf{W}_Q = \mathbf{P}\mathbf{X}\mathbf{W}_Q = \mathbf{P}\mathbf{Q}, \quad \mathbf{K}_{\text{perm}} = \mathbf{P}\mathbf{K}, \quad \mathbf{V}_{\text{perm}} = \mathbf{P}\mathbf{V}$$

Ma trận điểm số chú ý trước Softmax:

$$\frac{\mathbf{Q}_{\text{perm}} \mathbf{K}_{\text{perm}}^\top}{\sqrt{d}} = \frac{(\mathbf{P}\mathbf{Q})(\mathbf{P}\mathbf{K})^\top}{\sqrt{d}} = \mathbf{P} \left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}} \right) \mathbf{P}^\top$$

Áp dụng bổ đề cho trước $\text{softmax}(\mathbf{P}\mathbf{A}\mathbf{P}^\top) = \mathbf{P} \text{softmax}(\mathbf{A}) \mathbf{P}^\top$, ta tính toán ma trận trung gian $\mathbf{H}_{\text{perm}}$:

$$\mathbf{H}_{\text{perm}} = \text{softmax} \left(\mathbf{P} \frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}} \mathbf{P}^\top \right) \mathbf{V}_{\text{perm}} = \mathbf{P} \text{softmax} \left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}} \right) \mathbf{P}^\top (\mathbf{P}\mathbf{V}) = \mathbf{P}\mathbf{H}$$

Giai đoạn 2: Lớp Feed-Forward mạng vị trí. Do thuộc tính $\mathbf{P}\mathbf{1} = \mathbf{1}$, ta có $\mathbf{1}\mathbf{b}_1 = \mathbf{P}(\mathbf{1}\mathbf{b}_1)$, dẫn đến:

$$\mathbf{H}_{\text{perm}} \mathbf{W}_1 + \mathbf{1}\mathbf{b}_1 = \mathbf{P}(\mathbf{H}\mathbf{W}_1 + \mathbf{1}\mathbf{b}_1)$$

Áp dụng tính chất phân phối của hàm chỉnh lưu $\text{ReLU}(\mathbf{PA}) = \mathbf{PReLU}(\mathbf{A})$ và nhân ma trận trọng số thứ hai $\mathbf{W}_2$ kết hợp cộng vector định thiên $\mathbf{1b}_2 = \mathbf{P}(\mathbf{1b}_2)$, ta thu được:

$$\mathbf{Z}_{\text{perm}} = \mathbf{PReLU}(\mathbf{HW}_1 + \mathbf{1b}_1)\mathbf{W}_2 + \mathbf{P}(\mathbf{1b}_2) = \mathbf{PZ} \quad \blacksquare$$

- ii. Hệ quả toán học $\mathbf{Z}_{\text{perm}} = \mathbf{PZ}$ khẳng định rằng cấu trúc Transformer tiêu chuẩn khi khuyết thiếu thông tin vị trí mang tính chất **bất biến theo hoán vị (permutation equivariance)**. Việc xáo trộn trật tự các token đầu vào chỉ đơn thuần làm xáo trộn các vị trí tương ứng ở đầu ra, khiến biểu diễn học sâu của một từ hoàn toàn độc lập với vị trí phân bố của nó. Điều này gây ra hạn chế nghiêm trọng trong xử lý văn bản vì trật tự từ mang ý nghĩa cú pháp và ngữ nghĩa cốt lõi (ví dụ câu “chó cắn người” khác biệt hoàn toàn “người cắn chó”, nhưng mô hình sẽ coi chúng như một túi từ giống nhau).

Question 2(b) — Position embeddings

Xét hệ thống mã hóa vị trí hình sin:

$$\Phi_{(t,2i)} = \sin\left(\frac{t}{10000^{2i/d}}\right), \quad \Phi_{(t,2i+1)} = \cos\left(\frac{t}{10000^{2i/d}}\right), \quad \mathbf{X}_{\text{pos}} = \mathbf{X} + \Phi$$

- i. **Có**, cấu trúc mã hóa vị trí Φ giải quyết triệt để nhược điểm bất biến hoán vị ở phần (a). Do các vector Φ được tính toán cố định dựa trên chỉ số vị trí tuyệt đối t chứ không theo token, khi ta tiến hành hoán vị ma trận từ vựng, ta có mối quan hệ bất đối xứng $\mathbf{PX}_{\text{pos}} = \mathbf{PX} + \mathbf{P}\Phi \neq (\mathbf{PX}) + \Phi$. Sự lệch pha này bẻ gãy tính chất $\mathbf{Z}_{\text{perm}} = \mathbf{PZ}$, giúp mạng chú ý phân biệt được ngữ nghĩa dựa trên trật tự vị trí và cấu trúc chuỗi văn bản.
- ii. **Không**, trên tập hợp các vị trí số nguyên phân biệt, không thể tồn tại hai vị trí khác nhau có cùng một vector mã hóa vị trí Φ . Để chứng minh, ta xét riêng cặp chiều tần số cao nhất ứng với chỉ số $i = 0$: $(\Phi_{(t,0)}, \Phi_{(t,1)}) = (\sin t, \cos t)$. Đây là tọa độ của một điểm di chuyển trên đường tròn đơn vị trong không gian hai chiều, đặc trưng bởi góc quay t tính bằng radian. Hai vị trí số nguyên khác nhau $t \neq t'$ chỉ có thể trùng khít tọa độ khi hiệu số giữa chúng là một bội số nguyên của chu kỳ vòng tròn: $t - t' = 2m\pi$ ($m \in \mathbb{Z} \setminus \{0\}$). Tuy nhiên, do 2π là một số vô tỷ, phương trình này không tồn tại nghiệm nguyên cho cặp (t, t') . Vì chỉ riêng cặp chiều đầu tiên đã đủ khả năng phân tách mọi số nguyên t , toàn bộ vector mã hóa vị trí $\Phi_{(t,\cdot)}$ là duy nhất cho mỗi tọa độ thời gian.

3. Coding a Transformer from Scratch (30 points)

Toàn bộ hệ thống mã nguồn được hiện thực hóa trong tệp `model_solution.py`. Tóm tắt kiến trúc và phương thức triển khai các tầng chức năng theo đúng trình tự phụ thuộc:

- `MLP.forward` (1đ): Áp dụng tuần tự $\text{fc1} \rightarrow \text{GELU} \rightarrow \text{fc2}$, hiện thực cấu trúc mạng nơ-ron đa tầng: $\text{MLP}(\mathbf{x}) = \mathbf{W}_2 \text{GELU}(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2$, trong đó không gian ẩn được mở rộng lên mức $4d_{\text{model}}$.
- `CausalAttention.forward` (6đ): Chiếu tuyến tính ma trận đầu vào sang không gian $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ rồi phân tách thành cấu trúc đa đầu n_{heads} bằng toán tử `einops.rearrange` theo sơ đồ quy ước “`b t (h d) -> b h t d`”. Tiến hành tính toán ma trận điểm số chú ý thông qua

tích $\mathbf{Q}\mathbf{K}^\top/\sqrt{d_{\text{attn}}}$, áp dụng cơ chế mặt nạ nhân quả (`causal_mask`) bằng cách điền $-\infty$ vào các vị trí tương lai, lấy `softmax`, nhân tích chập với $\mathbf{V}$, gộp các đầu chú ý và chiếu qua ma trận đầu ra $\mathbf{W}_o$.

- `DecoderBlock.forward` (2đ): Khối giải mã áp dụng kết nối tắt dư kết hợp chuẩn hóa tiền tầng (Pre-Layer Normalization) theo quy trình toán học:

$$\mathbf{x} \leftarrow \mathbf{x} + \text{Attention}(\text{LN}_{\text{pre}}(\mathbf{x})), \quad \mathbf{x} \leftarrow \mathbf{x} + \text{MLP}(\text{LN}_{\text{post}}(\mathbf{x}))$$

- `Transformer.forward` (6đ): Cộng ma trận nhúng từ vựng với ma trận vị trí (`arange(seq_len)`), truyền qua chồng các khối `DecoderBlock`, xử lý qua tầng chuẩn hóa cuối `final_layer_norm`, rồi chiếu qua đầu phân loại `lm_head` để thu được logits có kích thước quy ước `[batch, seq_len, vocab_size]`.
- `Transformer.generate` (5đ): Triển khai thuật toán giải mã tham lam (*greedy decoding*), lặp lại `num_new_tokens` lần, tại mỗi bước cắt lát ngữ cảnh về tối đa `context_length`, trích xuất logits tại vị trí thời gian cuối cùng, áp dụng toán tử `argmax` để chọn ra token có xác suất cao nhất rồi nối vào chuỗi.
- `Transformer.get_loss_on_batch` (7đ): Tính toán ma trận logits và thực hiện dịch chuyển một bước thời gian phục vụ tác vụ dự đoán token kế tiếp, tính toán số mất mát bằng hàm entropy chéo (`cross_entropy`) đối chiếu giữa phân đoạn `logits[:, :-1]` và nhãn mục tiêu `input_ids[:, 1:]`.

Kết quả kiểm thử đơn vị (Part a). Hệ thống hoàn thành mã nguồn chính xác, vượt qua tuyệt đối bài kiểm tra gồm 6 unit tests độc lập:

```
..... 6 passed in 14.81s
```

Bao gồm các hàm test thành phần: `test_mlp`, `test_attention`, `test_decoder_block`, `test_forward`, `test_generate`, và `test_loss_on_batch`, xác thực sai số so với snapshot tham chiếu đạt mức lý tưởng ($< 10^{-5}$).

Huấn luyện mô hình (Part b iii). Thực hiện kích hoạt tệp lệnh `train.py` cấu hình mô hình thu nhỏ ($d_{\text{model}} = 33$, 3 đầu chú ý, 3 tầng decoder), thiết lập tốc độ học cơ sở $\eta = 10^{-5}$, kích thước batch bằng 16 và chạy trong vòng 100 bước tối ưu. Đồ thị hàm mất mát ghi nhận xu hướng **giảm đều và ổn định** từ mức ≈ 10.835 xuống mức ≈ 10.766 , xác nhận vòng lặp lan truyền ngược hoạt động chính xác theo đúng thiết kế toán học (Hình 1).

[Đồ thị `losses_and_grad_norms.png`: Loss huấn luyện (trái) và chuẩn Gradient (phải) qua 100 bước]

Hình 1: Đường cong hàm mất mát loss (trái) và chuẩn của gradient (phải) qua 100 bước huấn luyện trên cấu hình baseline.

Bonus (Part c) — Tăng tốc độ học với 100 bước cố định. Nhằm tối ưu hóa tốc độ hội tụ của hàm mất mát trong giới hạn nghiêm ngặt 100 bước lặp, tôi triển khai cấu hình cải tiến tích lũy cộng dồn gồm ba thay đổi kiến trúc và tham số, kiểm thử ghi nhận mỗi thay đổi đều đóng góp làm giảm mạnh giá trị loss cuối cùng:

1. **Thay đổi 1 (Tối ưu hóa tham số): Tăng tốc độ học hệ thống η từ $10^{-5} \rightarrow 10^{-3}$.** Tốc độ học cơ sở quá nhỏ khiến mô hình gần như chưa kịp học trong 100 bước; việc nâng cao η giúp tăng biên độ dịch chuyển gradient, mang lại mức suy giảm loss mạnh mẽ nhất.
2. **Thay đổi 2 (Kiến trúc mạng): Mở rộng kích thước không gian ẩn d_{model} từ 33 $\rightarrow$ 132.** Việc gia tăng số chiều giúp nâng cao dung lượng biểu diễn và mở rộng không gian lưu trữ thông tin ngữ nghĩa, cho phép mạng nơ-ron khớp dữ liệu tối ưu hơn trong cùng một khoảng thời gian.
3. **Thay đổi 3 (Kiến trúc mạng): Nâng cấp số đầu chú ý kết hợp mở rộng không gian ẩn (n_{heads} từ 3 $\rightarrow$ 6, d_{model} từ 132 $\rightarrow$ 264).** Việc tích hợp cấu hình đa đầu giúp mô hình song song hóa quá trình học tập trên nhiều không gian con chú ý khác nhau kết hợp dung lượng lớn hơn, hạ thấp hàm mất mát thêm một bậc đáng kể.

Ghi chú thực nghiệm: Tôi đã tiến hành thử nghiệm các kỹ thuật tối ưu khác (bao gồm lịch trình suy giảm tốc độ học hình cos, gia tăng số lượng tầng mạng, ràng buộc trọng số weight tying, loại bỏ cắt clip gradient, hiệu chỉnh batch size), tuy nhiên trong phạm vi giới hạn ngặt nghèo 100 bước lặp, các phương pháp này không đem lại hiệu quả vượt trội so với hướng tập trung mở rộng **dung lượng kiến trúc mô hình**.

Cấu hình thực nghiệm (Tích lũy cộng dồn)	Loss sau 100 bước
Baseline ($\eta = 10^{-5}$, $d_{\text{model}} = 33$)	10.7652
+ Change 1: Tăng tốc độ học $\eta = 10^{-3}$	6.4586
+ Change 2: Mở rộng không gian ẩn $d_{\text{model}} = 132$	4.5721
+ Change 3: Tối ưu cấu hình đa đầu $d_{\text{model}} = 264$, $n_{\text{heads}} = 6$	4.3953

Bảng 1: Chỉ số hàm mất mát loss cuối cùng giảm dần qua các cấu hình tối ưu tích lũy. Giá trị tối ưu thấp nhất đạt được: **4.3953**.